Cierre: lunes, 5 de octubre de 2026, 18:00

MODELACIÓN Y DISEÑO DE SISTEMAS · PARCIALES 2 Y 3

Trabajo integrador: del caso OAS a un proyecto listo para implementar

Trabajo grupal · Todo lo que vimos en el año, aplicado a su propio caso

Durante el año fuimos construyendo, pieza por pieza, todo lo que necesita un equipo para arrancar a programar un sistema: requerimientos, backlog ágil, modelos UML, arquitectura, patrones de diseño, prototipos y usabilidad. En este trabajo lo juntamos todo, sobre **el caso de OAS que ya presentaron**.

> **No es un trabajo para empezar ahora.** Casi todo lo que se pide ya lo fueron haciendo durante el año, en las actividades y trabajos prácticos de cada bloque: el backlog en Azure Boards, los diagramas UML, el trabajo de patrones, la evaluación de usabilidad.
>
> Lo que queda es **reunirlo, completar lo que falte, hacerlo coherente con su caso y entregarlo**. Si un grupo tiene que arrancar de cero, va a estar muy ajustado con los tiempos.

> **La pregunta que tiene que contestar su entrega:** si mañana un equipo nuevo —o un agente de IA— abre su repositorio sin conocer el caso, *¿puede empezar a implementar?*
>
> No alcanza con que cada documento exista. Tiene que estar **completo**, ser **coherente con los demás** y estar **escrito para alguien que no estuvo en la empresa**.

PASO 1 · LO PRIMERO QUE TIENEN QUE HACER

Apenas creen el proyecto en Azure DevOps (ver el **Anexo**), suban a Moodle, una vez por grupo:

- el **nombre y apellido de todos los integrantes**;
- el **link al proyecto público de Azure DevOps**;
- el **mail del interlocutor** del grupo.

Es parte fundamental de la actividad: **sin ese link no hay corrección**, ni revisión anticipada, ni devolución.

Fechas

Fecha	Qué pasa
Vie 23/10	Presentación de la consigna. Arman el proyecto en Azure DevOps y suben a Moodle **integrantes, link y mail del interlocutor** (Paso 1).
Vie 30/10	**Primera entrega.** Repositorio y Boards completos. Suben a Moodle el link del proyecto.
Lun 2/11	**Devolución por mail** al interlocutor del grupo: el informe de completitud por documento, la lista de faltantes a corregir y las observaciones del docente.
Jue 5/11	**Entrega final y evaluación.** En clase, el docente hace **consultas a cada grupo sobre su trabajo**. Después se ponen las notas del Parcial 2 y del Parcial 3.

Vie 6/11	**Recuperatorio oral** de **uno** de los tres parciales (1, 2 o 3), el que corresponda a cada alumno. Se pregunta sobre el tema en el que le fue mal.

Se corrige lo que esté en la rama `main` a las **23:59** del día de cada entrega. Lo que se suba después no se mira.

Modalidad

- **Grupal**, con los mismos grupos de la cursada.
- **Caso:** el trabajo de campo de OAS del grupo. Si el grupo cambió de integrantes, usen el caso de uno de ellos y avísenlo en el README.
- **Todo vive en Azure DevOps**: el backlog en *Boards* y los documentos en *Repos*. No se entregan PDFs sueltos por Moodle.
- **Está permitido usar IA**, incluido BMAD. Pero la defensa y el conocimiento tienen que ser de ustedes: cada integrante tiene que poder explicar cualquier parte del trabajo. **"Porque lo dijo la IA" no es una respuesta válida.**

Qué tienen que hacer

Once entregables, agrupados en cuatro bloques. Al lado de cada uno se indica a qué parcial suma nota: `P2` o `P3`.

Bloque A · Del caso OAS a un backlog ágil

1 **Backlog ágil desde el trabajo de OAS.** `P3`

- **Visión del producto** en un párrafo, derivada del mandato de OAS.
- Al menos **3 épicas**, cada una con al menos **2 features**.
- Al menos **15 historias de usuario** en formato *Como <rol> quiero <acción> para <beneficio>*, que cumplan INVEST.
- Cada HU con **prioridad** (MoSCoW) y **estimación** en story points.
- **Trazabilidad con OAS:** cada épica indica de qué parte del trabajo de OAS sale (alcance del mandato, flujograma TO-BE, formulario, procedimiento, problema del diagnóstico). Usen la guía *Mapeo OAS → Scrum*.

2 **Plan de sprints.** `P3`

- Duración del sprint y **capacidad del equipo** supuesta (en story points), con el criterio usado.
- Al menos **3 sprints**, cada uno con su **objetivo** (sprint goal) en una oración y las HU que entran.
- La suma de puntos de cada sprint no supera la capacidad. Las dependencias entre HU se respetan (no se planifica algo antes de lo que necesita).
- Los sprints cargados como *Iterations* en Azure Boards.

Bloque B · Análisis y arquitectura

3 **Diagrama de contexto y diagrama de casos de uso.** `P2`

- **Contexto:** el sistema como caja negra, todos los actores y sistemas externos, y los flujos de información que entran y salen.
- **Casos de uso:** al menos **6 casos de uso** con sus actores, y relaciones *include/extend* donde correspondan (no forzadas).
- Una **tabla de trazabilidad** caso de uso ↔ historias de usuario: cada caso de uso dice qué HU lo implementan.

4 **Requerimientos no funcionales y arquitectura.** `P2`

- Al menos **5 RNF cuantificados**: atributo de calidad, escenario y métrica medible. "*El sistema debe ser rápido*" no es un RNF; "*el 95% de las búsquedas responde en menos de 2 segundos con 50 usuarios concurrentes*" sí.
- **Estilo arquitectónico elegido** (capas, monolito modular, hexagonal, microservicios, eventos, cliente-servidor, combinación…) y **por qué**, argumentado desde los RNF.
- **Diagrama de componentes** (o de alto nivel) y **diagrama de despliegue UML** (nodos, artefactos, protocolos).

5 **Al menos un ADR (registro de decisión de arquitectura).** P2

Con estas secciones:

- **Título y estado** (propuesto / aceptado / reemplazado).
- **Contexto:** qué problema hay que decidir y qué RNF lo condicionan (citados por su identificador).
- **Alternativas consideradas:** al menos dos, con sus ventajas y desventajas.
- **Decisión** tomada.
- **Consecuencias:** lo que se gana *y lo que se pierde*. Un ADR sin consecuencias negativas no es creíble.

Bloque C · Diseño

6 **Diagrama de clases de diseño con al menos 2 patrones de diseño.** P2

- Al menos **10 clases**, con atributos tipados, métodos con firma completa, multiplicidad en todas las asociaciones, e interfaces o clases abstractas marcadas.
- Al menos **2 patrones GoF**, en lo posible de categorías distintas. Por cada patrón:
  - qué problema *de su caso* resuelve;
  - qué clase cumple cada rol del patrón (por ejemplo: *Context* → *Pedido*, *Strategy* → *IPoliticaDescuento*);
  - por qué ese patrón y no otro;
  - qué pasaría si no lo usaran.
- Las clases tienen que ser coherentes con los casos de uso y las HU: si una clase no aparece en ningún flujo, sobra; si una HU necesita algo que no está, falta.

7 **Diagrama de estados y diagrama de secuencia.** P2

- **Estados:** de la entidad central del caso (un pedido, un turno, un expediente…), con eventos, condiciones y estados finales.
- **Secuencia:** de una de las HU del punto 8, a nivel de diseño. Los mensajes tienen que ser **métodos que existen** en el diagrama de clases, y tiene que mostrar al menos un camino alternativo o de error.

Bloque D · Listo para implementar

8 **Historias de usuario completas, listas para que las implemente un agente.** P3

La cantidad depende del tamaño del grupo:

- Grupo de **3 integrantes: 2 HU**.
- Grupo de **4, 5 o 6 integrantes: 4 HU**.

Elíjanlas del backlog; al menos una tiene que usar alguno de sus patrones. Cada una en su propio archivo, con el formato de story que vimos en BMAD:

- Historia (*Como / quiero / para*), contexto y reglas de negocio que aplican.
- Al menos **4 criterios de aceptación** en formato *Dado / Cuando / Entonces*, de los cuales **al menos uno es un caso de error**.
- **Qué queda fuera** de la historia.
- **Notas de diseño:** qué clases del punto 6 toca y el **link a las capturas finales de sus pantallas** (punto 10), ya corregidas después de la evaluación heurística.
- **Datos de prueba** concretos para verificar cada criterio (los usan los casos de prueba del punto 9).
- Dependencias, estimación y chequeo de **Definition of Ready**.

9 **Plan de testing y casos de prueba de las HU del punto 8.** P3

Plan de testing (uno para todo el proyecto):

- **Alcance:** qué se prueba y qué no.
- **Niveles de la pirámide** (unitario, integración, E2E, aceptación del usuario): qué cubre cada uno *en su sistema* y en qué proporción.
- Al menos **una prueba ligada a un RNF** del punto 4 (rendimiento, seguridad, disponibilidad…), citado por su identificador.
- **Entornos y datos de prueba**, **herramientas**, y **cuándo corren** las pruebas (en cada commit, en el pipeline, antes de cada release).
- **Criterios de entrada y salida:** cuándo se empieza a probar y cuándo se da por aprobado.

Casos de prueba, por cada HU del punto 8:

- **Cada criterio de aceptación** tiene al menos un caso de prueba.
- Cada caso con: identificador, criterio que cubre, nivel (unitario, integración, E2E), precondiciones, pasos, datos de entrada y resultado esperado.
- Al menos un caso **negativo** (error o dato inválido) y uno de **valor límite** por historia.
- Una **matriz de trazabilidad** criterio de aceptación ↔ caso de prueba.

10 **Prototipo de pantallas y evaluación heurística.** P3

Se hace **por cada HU del punto 8**, en una sección propia del documento. Para cada historia:

- **Prototipo** de media fidelidad: al menos **2 pantallas**, y al menos una muestra un **estado de error** o un mensaje de validación. Herramienta libre (Figma, Penpot, Balsamiq, draw.io…). En el repo van las capturas y el link al prototipo.
- **Evaluación heurística** de esas pantallas con las 21 reglas (Shneiderman, Nielsen, Pierotti): al menos **3 heurísticas**, cada una con la pantalla, si cumple o no, la evidencia (captura señalada) y la severidad de 0 a 4.
- Al menos **1 corrección aplicada**, mostrando el antes y el después.

11 **Proyecto público en Azure DevOps.** P3

- **Boards:** la jerarquía Epic → Feature → User Story cargada, con los sprints como Iterations. Las HU del punto 8 con su descripción y criterios de aceptación **completos dentro del work item**.
- **Repos:** todos los documentos y diagramas, con la estructura de abajo.
- Proyecto con **visibilidad pública** y el docente agregado como usuario con el mail **bonofernando@hotmail.com**. Los pasos están en el **Anexo** al final.

Cómo se organiza el repositorio

La corrección arranca por el `README.md`. Si un documento no está indexado ahí, para la corrección **no existe**.

```
README.md                        ← índice de la entrega (obligatorio)
docs/
  01-vision-y-backlog.md         ← punto 1
  02-plan-de-sprints.md          ← punto 2
  03-contexto-y-casos-de-uso.md  ← punto 3
  04-rnf-y-arquitectura.md       ← punto 4
  adr/
    ADR-001-<titulo-corto>.md    ← punto 5
  06-diseno-de-clases.md         ← punto 6
  07-estados-y-secuencia.md      ← punto 7
  historias/
    HU-XXX-<titulo-corto>.md     ← punto 8 (una por archivo)
  testing/
    plan-de-testing.md           ← punto 9
    casos-HU-XXX.md              ← punto 9 (uno por HU)
  10-prototipo-y-heuristicas.md  ← punto 10
diagramas/                       ← fuentes de todos los diagramas
prototipo/                       ← capturas de pantallas
```

El `README.md` tiene que tener:

- Integrantes del grupo y nombre del caso de OAS.
- **Mail del interlocutor del grupo:** una persona que recibe las devoluciones y las comparte con el resto.
- Link al Board del proyecto.

- Una tabla con **cada uno de los 11 puntos → archivo donde está → qué quedó pendiente**, si algo quedó.

> **Diagramas: que se puedan leer como texto.** Cada diagrama va como imagen *y* con su archivo fuente:
>
> - **Recomendado:** Mermaid (Azure DevOps lo dibuja dentro del markdown) o PlantUML (`.puml`).
> - Si usan draw.io, suban el `.drawio` además del PNG.
> - Una imagen sola, sin fuente, se corrige con menos detalle: el corrector no puede verificar nombres de clases ni métodos.

Cómo se corrige

1. Un **agente de IA** recorre el repositorio a partir del `README.md` y revisa cada documento contra esta consigna.
2. Por cada documento informa su **grado de completitud** y **qué le falta para poder implementarse**. También cruza documentos: clases que no aparecen en la secuencia, HU sin caso de uso, ADR que no cita ningún RNF, etc.
3. El **docente revisa ese informe**, lo completa con su propia lectura y lo manda el **2/11 por mail al interlocutor del grupo**: el informe del agente junto con las observaciones del docente.
4. El **5/11, en clase**, el docente hace consultas a cada grupo sobre su trabajo. Cualquier integrante puede tener que explicar por qué tomaron una decisión.
5. La **nota la pone el docente** el 5/11, con el trabajo entregado y las respuestas a esas consultas. El informe del agente es un insumo, no la nota.

> **Revisión anticipada, las veces que quieran.** Antes de cualquier entrega, el grupo puede pedir por el **grupo de WhatsApp** de la materia que el agente revise su repositorio. Les llega el mismo informe de completitud y faltantes que usamos para corregir, para que se ayuden a completar el trabajo. No tiene límite de pedidos ni afecta la nota.

Consejo práctico: antes de entregar, pásenle su propio repositorio a una IA y pregúntenle *"¿qué te falta para poder implementar la HU-XXX?"*. Es exactamente lo que vamos a hacer nosotros.

Rúbrica · Parcial 2 — Arquitectura y diseño con patrones

Criterio	Puntos
Contexto y casos de uso (punto 3): actores completos, casos de uso bien delimitados, trazabilidad con las HU.	10
RNF cuantificados (punto 4): medibles y relevantes para el caso.	10
Arquitectura (punto 4): estilo justificado *desde los RNF*, componentes y despliegue coherentes entre sí.	20
ADR (punto 5): alternativas reales, decisión argumentada, consecuencias positivas y negativas.	10
Diseño de clases y patrones (punto 6): diagrama completo, patrones aplicados a un problema real del caso, roles mapeados, alternativa descartada.	25
Estados y secuencia (punto 7): coherentes con el diagrama de clases y con la HU elegida.	10
Trabajo de patrones GoF · entrega: entrega del trabajo de patrones hecho en la cursada.	5
Trabajo de patrones GoF · presentación: exposición del trabajo de patrones en clase.	10
Total	**100**

Rúbrica · Parcial 3 — Completitud de la materia

Criterio	Puntos
Backlog ágil desde OAS (punto 1): jerarquía completa, HU que cumplen INVEST, trazabilidad con el trabajo de OAS.	15
Plan de sprints (punto 2): objetivos claros, capacidad respetada, dependencias respetadas.	10
HU completas (punto 8: 2 o 4 según el grupo): criterios de aceptación verificables, caso de error, datos de prueba, notas de diseño.	15

Plan de testing y casos de prueba (punto 9): niveles justificados, prueba de un RNF, cada criterio de aceptación cubierto, casos negativos y de valor límite.	10
Prototipo (punto 10): pantallas de cada HU del punto 8, con estados de error.	10
Evaluación heurística (punto 10): por cada HU, argumentada desde la heurística, con evidencia y una corrección aplicada.	10
Azure DevOps (punto 11): Boards con jerarquía e iteraciones, repo ordenado, README completo.	10
Coherencia entre documentos: los mismos nombres, actores, clases y reglas en todos los artefactos.	10
Listo para implementar: un equipo o un agente que no conoce el caso puede empezar con las HU del punto 8 sin tener que preguntar.	10
Total	**100**

Lo que hace que un trabajo no apruebe

- Documentos genéricos que podrían ser de cualquier sistema: no se nota el caso de OAS.
- Patrones "puestos" en el diagrama sin un problema concreto que resuelvan.
- Una arquitectura elegida sin relación con los RNF.
- Criterios de aceptación que no se pueden verificar ("*el sistema funciona correctamente*").
- Artefactos que se contradicen: la clase se llama de una forma en el diagrama de clases y de otra en la secuencia, el actor de los casos de uso no existe en las HU, etc.

Qué se sube a Moodle

Primero, apenas crean el proyecto: **integrantes, link al proyecto de Azure DevOps y mail del interlocutor** (Paso 1). Después, una vez por grupo, el 30/10 y de nuevo el 5/11, el mismo link para confirmar la entrega. Todo lo demás se corrige desde el repositorio.

ANEXO

Cuenta de Azure, proyecto en Azure DevOps y acceso del docente

> La cuenta **Azure for Students** es gratuita y **no pide tarjeta de crédito** para validarse. La cuenta personal gratuita sí la pide. Por eso usen la de estudiante.

Paso 1 · Crear la cuenta de estudiante

1. Entren a azure.microsoft.com/en-us/free/students y hagan clic en **Start free**.
2. Inicien sesión con **cualquier mail**. Si no tienen cuenta de Microsoft, elijan "*¿No tiene una cuenta? Cree una*".
3. Si el mail ya se usa en más de una cuenta de Microsoft, les va a preguntar cuál usar: elijan la que prefieran y sigan.
4. Microsoft les manda un **código al mail**: escríbanlo para verificarlo.
5. Después les pide el **mail universitario** para verificar que son estudiantes. Ahí también llega un código.
6. Al terminar entran al portal de Azure, en la página **Education**, con el crédito de estudiante sin cargo.

Si un mail ya tenía una cuenta de Azure creada de antes, puede que no los deje activar la de estudiante. En ese caso prueben con otro mail en el punto 2. En el aula virtual está el video con el paso a paso.

Paso 2 · Crear la organización y el proyecto en Azure DevOps

1. Entren a aex.dev.azure.com con la misma cuenta y hagan clic en **Create new organization**. Basta con que **un integrante** cree la organización.
2. Para poder crear un proyecto público, primero habiliten esa opción en la organización: **Organization settings** → **Policies** → **Allow public projects** en *On*.
3. Creen el proyecto con **New project**, con **Visibility: Public** y proceso **Agile**. Así la jerarquía queda Epic → Feature → User Story.
4. Agreguen al resto del grupo como usuarios, igual que al docente en el paso 3.

Paso 3 · Agregar al docente

1. En dev.azure.com, elijan su organización.
2. Abajo a la izquierda, entren a **Organization settings**.
3. Elijan **Users** y hagan clic en **Add users**.
4. Completen así:
   - **Users:** bonofernando@hotmail.com
   - **Access level:** `Basic`. Si ya no quedan lugares Basic, usen `Stakeholder`.
   - **Add to projects:** el proyecto del trabajo.
   - **Azure DevOps Groups:** `Project Contributors`.
5. Hagan clic en **Add**.

> **Ojo con los lugares gratuitos.** El plan gratuito tiene **5 usuarios Basic**, contando a quien creó la organización. Los usuarios Stakeholder son ilimitados.
>
> En un proyecto **privado**, un Stakeholder **no ve el repositorio**. En un proyecto **público** sí puede verlo. Por eso el proyecto tiene que ser público: así entran todos los integrantes y el docente aunque el grupo sea de 5 o 6.

Material de apoyo: plantillas de Historia de Usuario, Criterios de Aceptación, DoR/DoD, Jerarquía Épica-Feature-HU y la guía *Mapeo OAS → Scrum*; las clases de Arquitectura y ADR, Patrones GoF, Testing, Usabilidad (las 21 reglas) y BMAD.

Agregar entrega

Estado de la entrega

Estado de la entrega	Todavía no se han realizado envíos
Estado de la calificación	Sin calificar
Tiempo restante	La Tarea está retrasada por: 18 días 21 horas